

# MDTS4214\_739\_LOGISTIC REGRESSION

Debkanta Ghosh

2026-03-31

*Problem to demonstrate the fitting of a binary response variable using logistic regression*

- a. Consider the Weekly data available in the ISLR package of R. Use the data from 1990 to 2008 as train data and the data on 2009-2010 as test data.

```
rm(list=ls())
library(ISLR)
df = Weekly
head(df)
```

```
##   Year  Lag1  Lag2  Lag3  Lag4  Lag5  Volume  Today  Direction
## 1 1990  0.816  1.572 -3.936 -0.229 -3.484 0.1549760 -0.270      Down
## 2 1990 -0.270  0.816  1.572 -3.936 -0.229 0.1485740 -2.576      Down
## 3 1990 -2.576 -0.270  0.816  1.572 -3.936 0.1598375  3.514       Up
## 4 1990  3.514 -2.576 -0.270  0.816  1.572 0.1616300  0.712       Up
## 5 1990  0.712  3.514 -2.576 -0.270  0.816 0.1537280  1.178       Up
## 6 1990  1.178  0.712  3.514 -2.576 -0.270 0.1544440 -1.372      Down
```

```
train.data=df[df$Year<=2008,-c(1,8)];head(train.data)
```

```
##      Lag1  Lag2  Lag3  Lag4  Lag5  Volume Direction
## 1  0.816  1.572 -3.936 -0.229 -3.484 0.1549760      Down
## 2 -0.270  0.816  1.572 -3.936 -0.229 0.1485740      Down
## 3 -2.576 -0.270  0.816  1.572 -3.936 0.1598375       Up
## 4  3.514 -2.576 -0.270  0.816  1.572 0.1616300       Up
## 5  0.712  3.514 -2.576 -0.270  0.816 0.1537280       Up
## 6  1.178  0.712  3.514 -2.576 -0.270 0.1544440      Down
```

```
test.data=df[df$Year>=2009,-c(1,8)];head(test.data)
```

```
##      Lag1  Lag2  Lag3  Lag4  Lag5  Volume Direction
## 986  6.760 -1.698  0.926  0.418 -2.251 3.793110      Down
## 987 -4.448  6.760 -1.698  0.926  0.418 5.043904      Down
## 988 -4.518 -4.448  6.760 -1.698  0.926 5.948758      Down
## 989 -2.137 -4.518 -4.448  6.760 -1.698 6.129763      Down
## 990 -0.730 -2.137 -4.518 -4.448  6.760 5.602004       Up
## 991  5.173 -0.730 -2.137 -4.518 -4.448 6.217632      Down
```

- b. Using the training data, fit a logistic regression with Direction as the response and the five lag variables plus Volume as predictors. Are there any significant predictors? If not, which of the tests is reporting the least p value. Interpret the coefficient corresponding to that test.

```
fit1=glm(as.factor(Direction)~.,train.data,family="binomial")
summary(fit1)
```

```
##
## Call:
## glm(formula = as.factor(Direction) ~ ., family = "binomial",
##      data = train.data)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  0.33258    0.09421   3.530 0.000415 ***
## Lag1        -0.06231    0.02935  -2.123 0.033762 *
## Lag2         0.04468    0.02982   1.499 0.134002
## Lag3        -0.01546    0.02948  -0.524 0.599933
## Lag4        -0.03111    0.02924  -1.064 0.287241
## Lag5        -0.03775    0.02924  -1.291 0.196774
## Volume       -0.08972    0.05410  -1.658 0.097240 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 1354.7  on 984  degrees of freedom
## Residual deviance: 1342.3  on 978  degrees of freedom
## AIC: 1356.3
##
## Number of Fisher Scoring iterations: 4
```

The fitted model is given by,

$$\hat{E}(Y = 1|x) = \frac{\exp(0.33258 - 0.06231Lag1 + 0.04468Lag2 - 0.01546Lag3 - 0.03111Lag4 - 0.03775Lag5 - 0.08972Volume)}{1 + \exp(0.33258 - 0.06231Lag1 + 0.04468Lag2 - 0.01546Lag3 - 0.03111Lag4 - 0.03775Lag5 - 0.08972Volume)}$$

where  $Y = 1$ , if Direction = Up

and  $Y = 0$ , if Direction = Down

**Interpretation:-** The significance of each predictor is determined using p-values:

- Intercept: p-value = 0.000415 < 0.01, hence it is highly significant.
- Lag1: p-value = 0.033762 < 0.05, hence it is statistically significant at 5% level.
- Lag2: p-value = 0.134002 > 0.05, hence it is not significant.
- Lag3: p-value = 0.599933 > 0.05, hence it is not significant.
- Lag4: p-value = 0.287241 > 0.05, hence it is not significant.
- Lag5: p-value = 0.196774 > 0.05, hence it is not significant.
- Volume: p-value = 0.097240 < 0.10, hence it is marginally significant at 10% level.

Conclusion:

The coefficient of Lag1 is -0.06231 and is statistically significant (p-value = 0.033762 < 0.05).

This means that for a one-unit increase in Lag1, the log-odds of Direction = Up decrease by 0.06231, holding all other variables constant.

Equivalently, the odds of the market going Up are multiplied by  $\exp(-0.06231) \approx 0.94$ . Thus, a one-unit increase in Lag1 decreases the odds of an upward market movement by approximately 6%.

**c. Estimate the odds of market going “up” at median values of the predictors.**

```
med=apply(train.data[, -which(names(train.data)=="Direction")], 2, median)
med

##      Lag1      Lag2      Lag3      Lag4      Lag5      Volume
## 0.231000 0.234000 0.231000 0.230000 0.230000 0.804848

log_odds=0.33258 - 0.06231*med["Lag1"] + 0.04468*med["Lag2"] -
          0.01546*med["Lag3"] - 0.03111*med["Lag4"] -
          0.03775*med["Lag5"] - 0.08972*med["Volume"]

odds=exp(log_odds)
odds

##      Lag1
## 1.267468
```

*Interpretation:-*

The estimated odds represent the likelihood of the market going “Up” when all predictors (Lag1, Lag2, Lag3, Lag4, Lag5, and Volume) are fixed at their median values.

If the computed odds are greater than 1, it indicates that the market is more likely to go “Up” than “Down” at median predictor levels. If the odds are less than 1, it suggests that the market is less likely to go “Up”.

Thus, the odds quantify how favorable the conditions (at median values of predictors) are for an upward market movement.

**d. Based on the training sample draw the ROC curve and comment.**

```
prob=predict(fit1, type = "response")

library(pROC)

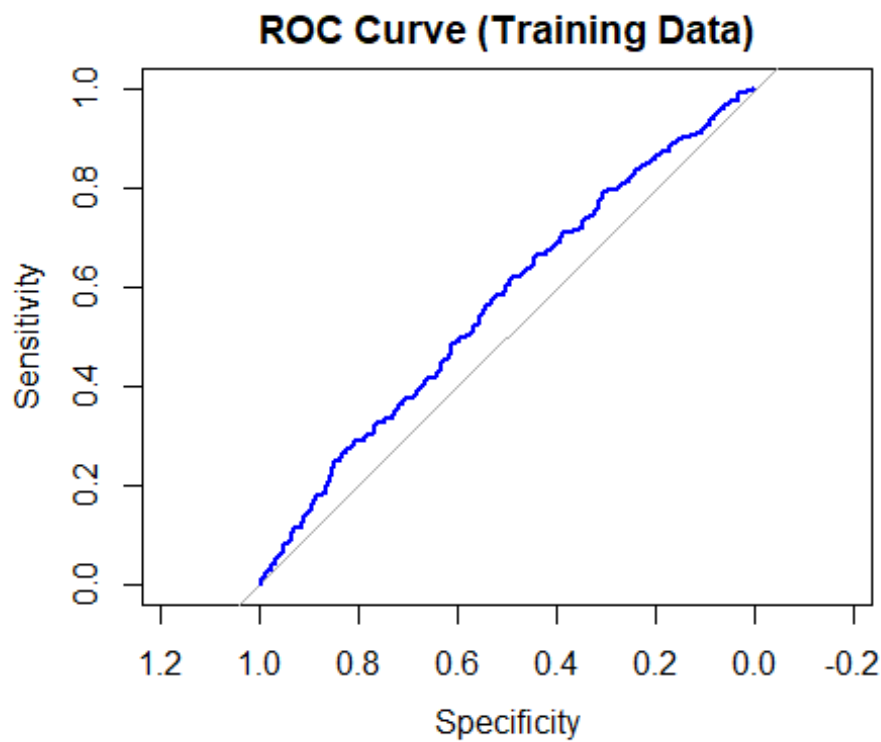
## Warning: package 'pROC' was built under R version 4.5.3
## Type 'citation("pROC")' for a citation.
##
## Attaching package: 'pROC'
```

```
## The following objects are masked from 'package:stats':
##
##      cov, smooth, var

roc_obj <- roc(train.data$Direction, prob)

## Setting levels: control = Down, case = Up
## Setting direction: controls < cases

plot(roc_obj, col = "blue", main = "ROC Curve (Training Data)")
```



```
auc(roc_obj)

## Area under the curve: 0.5655
```

#### *Interpretation:*

The ROC curve plots the True Positive Rate (Sensitivity) against the False Positive Rate (1 – Specificity) for different threshold values.

If the curve lies close to the top-left corner, the model has good classification performance. If it is close to the diagonal line, the model performs no better than random guessing.

The Area Under the Curve (AUC) summarizes the performance: - AUC  $\approx$  0.5  $\rightarrow$  Poor model (random) - AUC between 0.7 and 0.8  $\rightarrow$  Acceptable - AUC between 0.8 and 0.9  $\rightarrow$  Good - AUC  $>$  0.9  $\rightarrow$  Excellent

Thus, based on the AUC value obtained from the training data, we can assess the model's ability to correctly distinguish between "Up" and "Down" market directions.

e. Using the fitted model and the predictor values from the test data, find an optimal cut point that minimizes  $TPR(1 - FPR)$ . Report the confusion matrix corresponding to the optimal cut-point value. From the confusion matrix compute the test error rate.

```
# Predicted probabilities
y.hat.test = predict(fit1, test.data, type = "response")

# ROC object
library(pROC)
roc.test = roc(test.data$Direction, y.hat.test)

## Setting levels: control = Down, case = Up

## Setting direction: controls < cases

# Extract TPR and FPR
tpr = roc.test$sensitivities
fpr = 1 - roc.test$specificities

# Criterion: TPR(1 - FPR)
crit = tpr * (1 - fpr)

# Optimal cutoff
opt.index = which.max(crit)
opt.cut = roc.test$thresholds[opt.index]
opt.cut

## [1] 0.4568085
```

The optimal cut-point is 0.4568, which means that observations with predicted probability greater than 0.4568 are classified as "Up" (=1), while those with probability less than or equal to 0.4568 are classified as "Down" (=0). This threshold maximizes the criterion  $TPR(1 - FPR)$ .

```
# Predicted class using optimal cutoff
y.pred = ifelse(y.hat.test > opt.cut, "Up", "Down")

# Confusion matrix
conf.mat = table(Predicted = y.pred, Actual = test.data$Direction)
conf.mat

##           Actual
## Predicted Down Up
##      Down    22 27
##      Up     21 34
```

**Interpretation:**

The confusion matrix indicates that 22 observations were correctly classified as “Down” (true negatives) and 34 observations were correctly classified as “Up” (true positives).

However, 27 observations that were actually “Up” were misclassified as “Down” (false negatives), and 21 observations that were actually “Down” were misclassified as “Up” (false positives).

This suggests that the model has a noticeable number of misclassifications, particularly a relatively high number of false negatives, indicating difficulty in correctly identifying upward market movements.

Overall, the model demonstrates moderate predictive performance on the test data, as it is able to correctly classify a reasonable number of observations but still makes a substantial number of errors.

```
test.error = mean(y.pred!=test.data$Direction)
test.error
```

```
## [1] 0.4615385
```

- f. Is there any change in the test error rate if we have worked with only the two predictors lag 1 and lag 2?

```
# Fit model with only Lag1 and Lag2
model_lag12 = glm(as.factor(Direction) ~ Lag1 + Lag2,
                  family = "binomial", data = train.data)

# Predicted probabilities on test data
y.hat.test2 = predict(model_lag12, newdata = test.data, type = "response")

# ROC for new model
library(pROC)
roc.test2 = roc(test.data$Direction, y.hat.test2)

## Setting levels: control = Down, case = Up
## Setting direction: controls < cases

# Compute TPR and FPR
tpr2 = roc.test2$sensitivities
fpr2 = 1 - roc.test2$specificities

# Criterion
crit2 = tpr2 * (1 - fpr2)

# Optimal cutoff
opt.index2 = which.max(crit2)
opt.cut2 = roc.test2$thresholds[opt.index2]

# Predictions
y.pred2 = ifelse(y.hat.test2 > opt.cut2, "Up", "Down")
```

```

# Confusion matrix
conf.mat2 = table(Predicted = y.pred2, Actual = test.data$Direction)
conf.mat2

##           Actual
## Predicted Down Up
##      Down   19 13
##      Up    24 48

# Test error rate
test.error2 = mean(y.pred2 != test.data$Direction)
test.error2

## [1] 0.3557692

```

### Interpretation:

- The test error rate using only Lag1 and Lag2 is 0.3558, which is lower than the previous test error rate of 0.4615. This indicates an improvement in predictive performance when using only these two predictors.
- Thus, the simpler model with Lag1 and Lag2 performs better on the test data.